

Diseño de casos de *test*

Definición de casos de prueba

El objetivo del diseño de la prueba es entender que cada ensayo consume tiempo y dinero.

Por lo tanto:

- Debo encontrar la mayor cantidad de defectos en la menor cantidad de ensayos.
- Cada ensayo debe ser lo más breve posible.

Las **condiciones de prueba** son descripciones de *situaciones* ante las que el sistema debe responder que quieren probarse.

Los **casos de prueba** son lotes de datos necesarios para que se dé una determinada condición de prueba.

Podemos decir que un programa será correcto:

- Si para todas las combinaciones de datos de entrada válidos la salida es correcta.
- Cada combinación será un caso de prueba.

La prueba exhaustiva es imposible de realizar.

Estrategia de pruebas

Criterio de selección:

- Una condición para seleccionar un conjunto de casos de prueba.

De todas las combinaciones posibles solo elegiremos algunas:

- La menor cantidad de aquellas que tengan mayor probabilidad de encontrar un defecto no encontrado por otra prueba.

Especificación de los tipos de pruebas a realizar.

Técnicas de pruebas estáticas (analíticas)

Analizar el producto para deducir su correcta operación.

- Análisis de código fuente.
- Análisis automatizado de código fuente.
- Análisis formal.

Análisis de código fuente

Esta actividad consiste en revisar el código buscando problemas en algoritmos, defectos y otras faltas y se lleva a cabo en grupos.

Las estrategias de trabajo en el análisis de código fuente son variadas:

- Revisión de escritorio.
- Recorridas e Inspecciones.
 - Criticar al producto y no a la persona.
 - Permiten:
 - Unificar el estilo de programación.
 - Igualar hacia arriba la forma de programar.
 - No deben usarse para evaluar a los programadores.

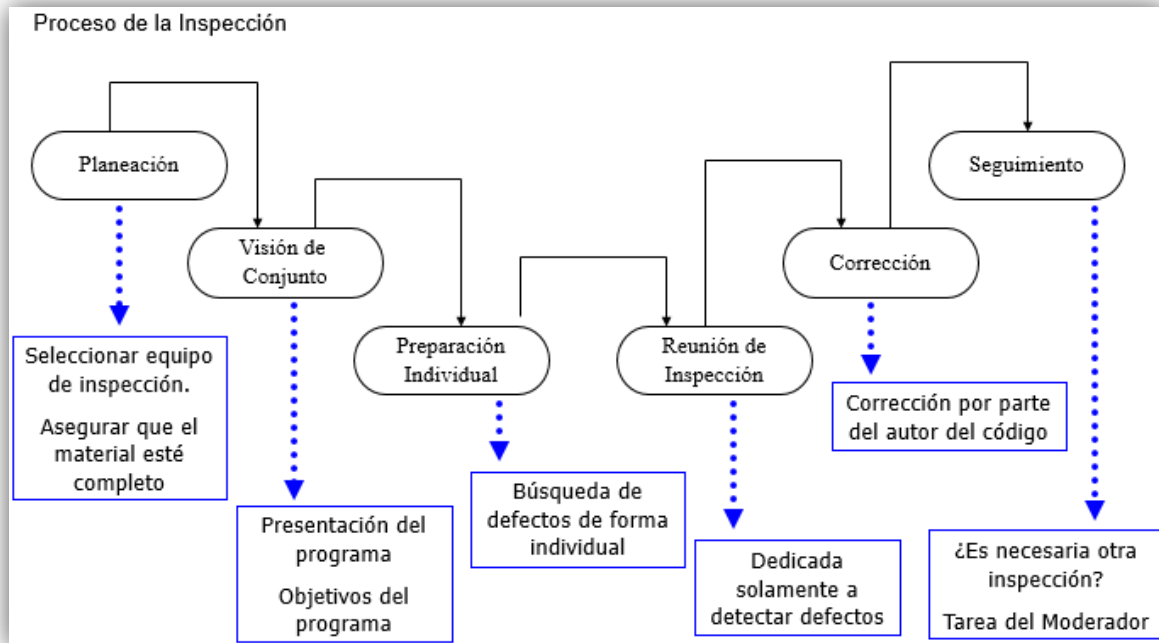
Recorridas

Esta actividad simula la ejecución de código para descubrir faltas con un número reducido de personas. Los participantes reciben antes el código fuente. Sus reuniones no duran más de dos horas. El foco está en detectar faltas y no en corregirlas. Los roles clave son: autor: presenta y explica el código; moderador: organiza la discusión; y secretario: escribe el reporte de la reunión para entregarle al autor. El autor es el que se encarga de la “ejecución” del código.

Inspecciones

Se examina el código (no solo aplicables al código) buscando faltas comunes. Se usa una lista de faltas comunes (*check-list*). Estas listas dependen del lenguaje de programación y de la organización. Por ejemplo, revisan: uso de variables no inicializadas, asignaciones de tipos no compatibles. Los roles clave son: moderador o encargado de la inspección, secretario, lector, inspector y autor. Todos son inspectores. Es común que una persona tenga más de un rol. Algunos estudios indican que el rol del lector no es necesario.

Proceso de la inspección



Análisis automatizado de código fuente

La entrada es el código fuente del programa y la salida es una serie de defectos detectados.

Herramientas de software que recorren código fuente y detectan posibles anomalías y faltas.

No requieren ejecución del código a analizar.

Es mayormente un análisis sintáctico:

- Instrucciones bien formadas
- Inferencias sobre el flujo de control

Complementan al compilador

Ejemplo:

- `a := a + 1` El analizador detecta que se asigna dos veces
- `a := 2` La variable sin usarla entre las asignaciones

Análisis formal

Se parte de una especificación formal y se busca probar (demostrar) que el programa cumple con la misma.

Un programa es correcto si cumple con la especificación.

Se busca demostrar que el programa es correcto a partir de una especificación formal.

Objeciones:

- Demostración más larga (y compleja) que el propio programa.
- La demostración puede ser incorrecta.
- Demasiada matemática para el programador medio.
- No se consideran limitaciones del hardware.
- La especificación puede ser incorrecta → igual hay que validar mediante pruebas. Esto ocurre siempre (*nada es 100% seguro*).

Que sea la máquina la que demuestre.

Desarrollar herramientas que tomen:

- Especificación formal.
- Código del componente a verificar.

Responda al usuario:

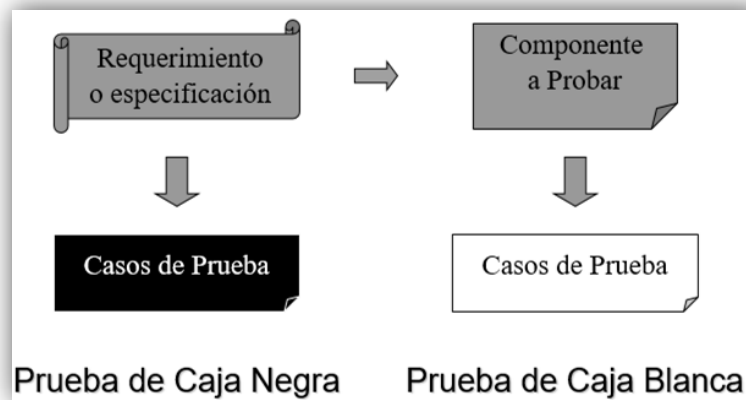
- El componente es correcto o
- Muestre un contraejemplo para mostrar que la entrada no se transforma de forma adecuada en la salida

Esta herramienta no puede ser construida (problema de la parada) → *Demostración asistida*.

Técnicas dinámicas (pruebas)

Experimentar con el comportamiento de un producto para ver si el producto actúa como es esperado.

- Caja blanca.
- Caja negra.

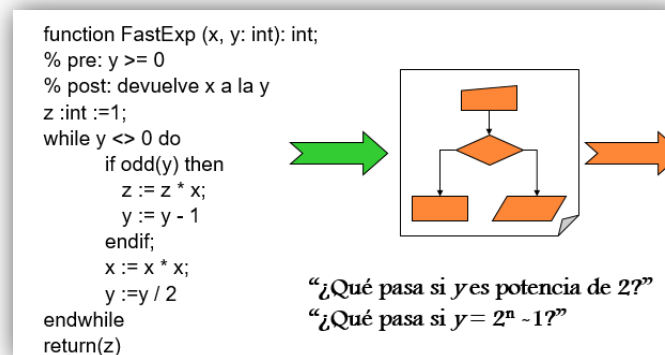


Caja blanca

A partir del código identificar los casos de prueba interesantes:

- Casos de prueba – Se necesita disponer del código.
- Tiene en cuenta las características de la implementación.
- Puede llevar a soslayar algún requerimiento no implementado.

Pruebas de caja blanca



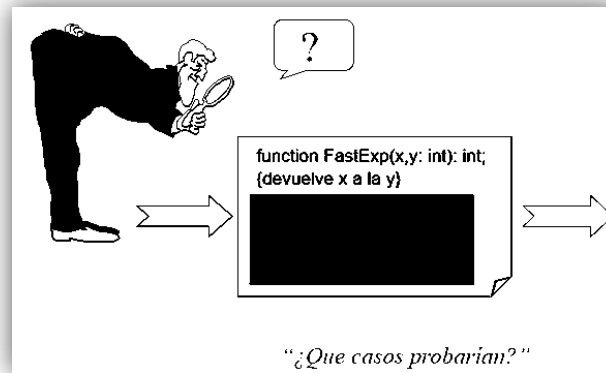
- Prueba estructural.
- Prueba lo que el software hace.
- Los casos de prueba se definen a partir de la estructura interna del programa.

Caja negra

Entrada a una caja negra de la que no se conoce el contenido y ver qué salida genera.

- Casos de prueba - No se precisa disponer del código.
- Se parte de los requerimientos y/o especificación.
- Porciones enteras de código pueden quedar sin ejercitar.

Pruebas de caja negra



- Prueba funcional, producida por los datos, o producida por la entrada/salida.
- Prueba lo que el software debería hacer.

Se basa en la definición del módulo a probar (definición necesaria para construir el módulo) nos desentendemos completamente del comportamiento y estructura internos del programa.

Clase de equivalencia

Podemos suponer que la prueba de un valor representativo de cada clase es equivalente a la prueba de cualquier otro valor. Llamamos a cada subconjunto "Clase de equivalencia".

Estas pruebas son llamadas "Pruebas por partición de equivalencia" o "Pruebas basadas en subdominios".

Partición de equivalencia. ¿Cómo hacerlo?

El proceso incluye dos pasos: identificar las clases de equivalencia y definir casos de prueba.

La identificación de clases de equivalencia se hace dividiendo cada condición de entrada en dos grupos: clase válida y clase inválida.

Identificación de clases de equivalencia

Para cada condición de entrada:

- Rango de valores. Ejemplo: 100 ≤ Sucursal Nº ≤ 170. / una válida y dos inválidas.
- Valor. Ejemplo: Número de documento / una válida y dos inválidas.
- Conjunto de valores. Ejemplo: DNI, CI, LE / una válida y una inválida.
- “Lógica”. Ejemplo: Primera letra del código = “T” / una válida y una inválida.

Análisis de condiciones de borde

La experiencia muestra que los casos de prueba que exploran las condiciones de borde producen mejor resultado que aquellas que no lo hacen. Seleccionamos uno o más elementos de manera que cada margen de la clase de equivalencia sea sujeto de una prueba. Además de mirar las condiciones de entrada, podemos mirar las clases de equivalencia de salida. Ejemplo: listados. La creatividad juega un papel clave.

Condiciones de borde. ¿Cómo hacerlo?

Rango de valores / casos válidos para los extremos del rango, y casos inválidos para los valores siguientes a los extremos.

Número de valores / máximo, mínimo y uno por encima y debajo de esos valores.

Conjuntos de datos (archivos) / archivos vacíos, primer registro, último registro, tratamiento del fin de archivo.

Ejemplo: Ingreso de cliente

Apellido 30 caracteres, obligatorio

Nombres 30 caracteres, obligatorio

Documento 1

Tipo de documento 1 DNI, LC, LE, CI, CM, P, F

Número de documento 1 8 dígitos sin signo

Código de provincia 01 a 23 o 40

Estado civil S, C, V, D, O (opcional)

Cantidad de hijos 0 a 20 (opcional)

Tipo de IVA	RI, RNI, EX
Ingreso mensual	6 dígitos sin signo (opcional)
Tipo identificación DGI	CUIT, CUIL, CDI
Número inscripción DGI	11 dígitos sin signo

Guía para análisis de frontera

Establecer casos válidos para los límites de los conjuntos y valores justo después de esos límites. Si es un conjunto de valores, establecer como valores de prueba el mínimo y el máximo del conjunto y los primeros valores antes del mínimo y luego del máximo como no válidos.

Conjetura de errores

También llamada “Prueba de sospechas” / “Sospechamos” que algo puede andar mal: Enumeramos una lista de errores posibles o de situaciones propensas a tener errores y Creamos casos de prueba basados en esas situaciones.

Es un proceso muy efectivo que puede ser formalizado a partir del análisis de las fallas.

El programador es quien puede darnos información más relevante / Dos orígenes: partes complejas del programa y circunstancias del desarrollo.

¡No perdamos esta oportunidad!

Error guessing

Se basa en la intuición o experiencia sobre errores comunes en la organización. La historia de defectos es una gran ayuda.

Ejemplos comunes son: listas o valores vacíos, cero, números negativos, valores, significativos, valores con significado cultural, valores organizacionales.

Pairwise

Intenta encontrar “clases de equivalencia” de conjuntos de valores. Debido a la combinatoria de las entradas es altamente costoso establecer todas las combinaciones.

Por otro lado, muchas combinaciones son equivalentes. Se intenta establecer que combinaciones son equivalentes a otras y testearlas.

Comparación de las técnicas estática y dinámica

Estáticas (análisis)

- Efectivas en la detección temprana de defectos.
- Sirven para verificar cualquier producto (requerimientos, diseño, código, casos de prueba, etc.).
- Conclusiones de validez general.
- Sujeto a los errores de nuestro razonamiento.
- Normalmente basadas en un modelo del producto y no en el producto.
- No se usan para validación (solo verificación).
- Dependen de la especificación.
- No consideran el hardware o el software de base.

Dinámicas (pruebas):

- Se considera el ambiente donde es usado el software (realista).
- Sirven tanto para verificar como para validar.
- Sirven para probar otras características además de funcionalidad.
- Está atado al contexto donde es ejecutado.
- Su generalidad no es siempre clara (solo se aseguran los casos probados).
- Solo sirve para probar el software construido.
- Normalmente se detecta un único error por prueba (los errores cubren a otros errores o el programa colapsa).

Complejidad ciclomática

Métrica del software que proporciona una medición cuantitativa de la complejidad lógica de un programa:

- Cantidad de caminos independientes.

- Camino independiente: agrega un nuevo conjunto de sentencias de procesamiento o una nueva condición.

Formas de calcularla:

- Número de regiones del grafo.
- $V(g) = A - N + 2$ (A = aristas, N = nodos).
- $V(g) = P + 1$ (P = cantidad de nodos predicado).

Grafo de flujo resultante:

- Camino 1: 1-2-10-11-13
- Camino 2: 1-2-10-12-13
- Camino 3: 1-2-3-10-11-13
- Camino 4: 1-2-3-4-5-8-9-2...
- Camino 5: 1-2-3-4-5-6-8-9-2...
- Camino 6: 1-2-3-4-5-6-7-8-9-2...

